

Introducción a la Programación con Python

Semana 4: Librerías y Módulos

▽Nablabs

Contenido

- 1 ¿Qué son los módulos y paquetes?
- 2 Importación de módulos
- 3 Importación selectiva
- 4 Librería estándar de Python
- 5 Módulo random
- 6 Módulo statistics
- 7 Módulos sys y os
- 8 Uso de pip
- 9 Creación de módulos propios
- 10 Documentación y búsqueda
- 11 Ejemplo integrador

¿Qué son?

 **Módulo:** Archivo .py con funciones y variables reutilizables

 **Paquete:** Colección de módulos organizados en carpetas

 Permiten reutilizar código sin repetirlo

 Acceso a código creado por otros programadores

 ¡No reinventes la rueda!

Importar con import

¿Cómo funciona?

Trae todo el módulo y accedes con punto (.)

```
1 import math
2
3 # Usar funciones del módulo
4 raiz = math.sqrt(16)
5 print(f"Raíz cuadrada: {raiz}")
6
7 pi = math.pi
8 print(f"Valor de pi: {pi}")
```

➡ Salida

Raíz cuadrada: 4.0

Valor de pi: 3.141592653589793

i Siempre usa el prefijo del módulo: `math.sqrt()`

¿Para qué?

Acorta el nombre del módulo para escribir menos

```
1 import math as m
2 import statistics as stats
3
4 raiz = m.sqrt(25)
5 media = stats.mean([1, 2, 3, 4, 5])
6
7 print(f"Raíz: {raiz}")
8 print(f"Media: {media}")
```

➔ Salida

Raíz: 5.0

Media: 3

💡 Útil con nombres largos o convenciones (numpy as np)

from ... import ... 

¿Cómo funciona?

Importa solo funciones específicas sin necesidad del prefijo

```
1 from math import sqrt, pi, pow
2
3 # Usar directamente sin "math."
4 raiz = sqrt(16)
5 potencia = pow(2, 3)
6
7 print(f"Raíz: {raiz}")
8 print(f"Pi: {pi}")
9 print(f"Potencia: {potencia}")
```

➔ Salida

Raíz: 4.0

Pi: 3.141592653589793

Potencia: 8.0

Importar todo con * *

```
1 from math import *  
2  
3 # Todo disponible sin prefijo  
4 resultado = sqrt(9) + pi  
5 print(f"Resultado: {resultado}")
```

➔ Salida

Resultado: 6.141592653589793



Advertencia

No recomendado:

- Contamina el namespace
- Dificulta saber origen
- Puede causar conflictos

¿Qué es?

Colección de módulos que vienen incluidos con Python

Módulos comunes:

-  `random` - Aleatoriedad
-  `statistics` - Estadística
-  `sys` - Sistema
-  `os` - Sistema operativo

Otros útiles:

-  `datetime` - Fechas
-  `json` - Formato JSON
- `re` - Expresiones regulares
-  `urllib` - URLs

 ¡No necesitas instalar nada!

random: Números aleatorios

```
1 import random
2
3 # Número aleatorio entre 0 y 1
4 aleatorio = random.random()
5
6 # Entero aleatorio en rango
7 dado = random.randint(1, 6)
8
9 # Elegir elemento de lista
10 frutas = ["manzana", "banana", "cereza"]
11 fruta = random.choice(frutas)
12
13 print(f"Aleatorio: {aleatorio:.2f}")
14 print(f"Dado: {dado}")
15 print(f"Fruta: {fruta}")
```

➔ Salida (ejemplo)

Aleatorio: 0.73

Dado: 4

Fruta: banana

random: Más funciones

```
1 import random
2
3 # Mezclar lista
4 numeros = [1, 2, 3, 4, 5]
5 random.shuffle(numeros)
6 print(f"Mezclados: {numeros}")
7
8 # Seleccionar múltiples elementos
9 colores = ["rojo", "azul", "verde", "amarillo"]
10 seleccion = random.sample(colores, 2)
11 print(f"Selección: {seleccion}")
```

➡ Salida (ejemplo)

Mezclados: [3, 1, 5, 2, 4]
Selección: ['verde', 'rojo']

⚠ shuffle() modifica la lista original

```
1 import statistics as stats
2
3 datos = [10, 20, 30, 40, 50]
4
5 media = stats.mean(datos)
6 mediana = stats.median(datos)
7 desv_std = stats.stdev(datos)
8
9 print(f"Media: {media}")
10 print(f"Mediana: {mediana}")
11 print(f"Desviación estándar: {desv_std:.2f}")
```

➔ Salida

Media: 30.0

Mediana: 30

Desviación estándar: 15.81

💡 Ideal para análisis rápidos de datos

sys: Información del sistema >_

```
1 import sys
2
3 # Versión de Python
4 print(f"Versión: {sys.version}")
5
6 # Argumentos de línea de comandos
7 print(f"Argumentos: {sys.argv}")
8
9 # Plataforma
10 print(f"Plataforma: {sys.platform}")
11
12 # Salir del programa
13 # sys.exit()
```

➔ Salida (ejemplo)

Versión: 3.11.0

Argumentos: ['programa.py']

Plataforma: win32

```
1 import os
2
3 # Directorio actual
4 directorio = os.getcwd()
5 print(f"Directorio: {directorio}")
6
7 # Listar archivos
8 archivos = os.listdir(".")
9 print(f"Archivos: {archivos[:3]}") # Primeros 3
10
11 # Crear directorio
12 # os.mkdir("nueva_carpeta")
13
14 # Verificar si existe
15 existe = os.path.exists("archivo.txt")
16 print(f"Existe: {existe}")
```

➔ Salida (ejemplo)

Directorio: /home/usuario

Archivos: ['programa.py', 'datos.txt', ...]

¿Qué es pip?

Herramienta para instalar paquetes externos de PyPI (Python Package Index)

Comandos básicos en terminal:

```
# Instalar paquete
pip install requests
# Instalar versión específica
pip install numpy==1.24.0
# Desinstalar
pip uninstall requests
# Listar instalados
pip list
# Actualizar paquete
pip install --upgrade requests
```

Ejemplo: Usar paquete instalado

```
1 # Primero instalar: pip install requests
2 import requests
3
4 # Hacer petición HTTP
5 respuesta = requests.get("https://api.github.com")
6
7 print(f"Código: {respuesta.status_code}")
8 print(f"Tipo: {respuesta.headers['Content-Type']}")
```

➔ Salida

Código: 200

Tipo: application/json; charset=utf-8

i Más de 400,000 paquetes disponibles en PyPI

¿Cómo crear un módulo?

Simplemente crea un archivo .py con funciones

Archivo: `matematicas.py`

```
1 def sumar(a, b):  
2     """Suma dos números"""  
3     return a + b  
4  
5 def multiplicar(a, b):  
6     """Multiplica dos números"""  
7     return a * b  
8  
9 PI = 3.14159
```

 El nombre del archivo es el nombre del módulo

Archivo: programa.py (en la misma carpeta)

```
1 import matematicas
2
3 resultado = matematicas.sumar(5, 3)
4 print(f"Suma: {resultado}")
5
6 producto = matematicas.multiplicar(4, 6)
7 print(f"Producto: {producto}")
8
9 print(f"Pi: {matematicas.PI}")
```

➔ Salida

Suma: 8

Producto: 24

Pi: 3.14159

✓ ¡Así de fácil es crear y usar tus módulos!

Módulo con `if __name__ == "__main__":`

Archivo: calculadora.py

```
1 def sumar(a, b):
2     return a + b
3
4 def restar(a, b):
5     return a - b
6
7 # Solo se ejecuta si se corre directamente
8 if __name__ == "__main__":
9     print("Probando calculadora...")
10    print(f"5 + 3 = {sumar(5, 3)}")
11    print(f"5 - 3 = {restar(5, 3)}")
```

i El bloque no se ejecuta cuando importas el módulo

¿Dónde buscar librerías?

 **PyPI:** `pypi.org` - Repositorio oficial

 **Docs Python:** `docs.python.org` - Librería estándar

 **GitHub:** Código fuente y ejemplos

 **Stack Overflow:** Preguntas y respuestas

Más utilizadas en Python

-  **NumPy**: Computación numérica
-  **Pandas**: Análisis de datos
-  **Matplotlib**: Visualización de datos
-  **Requests**: Peticiones HTTP
-  **Flask/Django**: Desarrollo web
-  **Scikit-learn**: Machine Learning
-  **TensorFlow/PyTorch**: Deep Learning
- Beautiful Soup**: Web scraping

Archivo: notas_utils.py

```
1 import statistics
2
3 def calcular_promedio(notas):
4     """Calcula el promedio de las notas"""
5     return statistics.mean(notas)
6
7 def nota_maxima(notas):
8     """Retorna la nota máxima"""
9     return max(notas)
10
11 def nota_minima(notas):
12     """Retorna la nota mínima"""
13     return min(notas)
14
15 def estado(promedio):
16     """Determina si aprobó o reprobó"""
17     return "Aprobado" if promedio >= 60 else "Reprobado"
```

Archivo: analizar.py

```
1 import notas_utils as nu
2 import random
3 # Generar notas aleatorias para demostración
4 notas = [random.randint(40, 100) for _ in range(5)]
5 print(f"Notas: {notas}")
6 print(f"Promedio: {nu.calcular_promedio(notas):.1f}")
7 print(f"Máxima: {nu.nota_maxima(notas)}")
8 print(f"Mínima: {nu.nota_minima(notas)}")
9 promedio = nu.calcular_promedio(notas)
10 print(f"Estado: {nu.estado(promedio)}")
```

➡ Salida (ejemplo)

Notas: [85, 72, 91, 68, 77]

Promedio: 78.6

Máxima: 91

Mínima: 68

Estado: Aprobado

¿Qué aprendimos hoy?

- ✓ Módulos y paquetes
- ✓ Importar con `import` y `from ... import`
- ✓ Librería estándar de Python
- ✓ Módulos: `random`, `statistics`, `sys`, `os`
- ✓ Uso de `pip` para instalar paquetes
- ✓ Creación de módulos propios
- ✓ Documentación y búsqueda de librerías



¡Explora el ecosistema Python!



¡Preguntas!

Gracias por su atención